Symplectic Jump Generation: A Deterministic Geometric Engine for Semantic and Formal Language Domains

Sauli Kälkäjä[1] Claude (Anthropic)[2,†] Gemini (Google)[3,†]

Audit: Qwen (Alibaba)[4,†]

[1] Independent researcher, Finland [2] Anthropic [3] Google [4] Alibaba

[†] Large language model collaborators in a human-AI authorship framing.

April 2026

Abstract

We report on Project Aisha, a deterministic cognitive engine that replaces stochastic next-token prediction with geometric jumps on a symplectic manifold of word vectors. Building on prior work that established the 8D Complex Hermitian Manifold for celestial and quantum mechanics under the symplectic lock $\alpha\beta = 1$ and the hyperbolic invariant $(\alpha + \beta)^2 - M^2 = 4$, we apply the same geometric machinery to two language domains: natural English and Python source code. We report *five empirical falsifications* on natural language showing that a symplectic jump by $\arg\max_v \langle \mathbf{E}_{\text{norm}}[v], \mathbf{c}_{\text{composed}} \rangle$ cannot produce grammatical sentences because function words are distributionally diffuse in any natural-text co-occurrence embedding. We then report *one positive empirical validation* on Python source: with three minimal additions (log-bigram reweighting, bracket-depth conservation, and a parser-based rejection filter), the same engine produces syntactically valid Python at a 62% parse rate on 100 randomly-sampled Python prefixes from the corpus, versus 0% for a unigram baseline. Without the parser-based rejection filter, the engine still produces valid Python at 30% — demonstrating that symplectic selection plus corpus-intrinsic statistics alone (no external parser) carry 30 percentage points of the full result. The engine runs at $\mathcal{O}(1)$ per token, uses $\sim$2.5 MB on disk, and requires no GPU. We argue this validates the symplectic framework as a *selection* mechanism whose effectiveness depends on the structural regularity of its substrate. All code, embeddings, and results are open source.

Contents

1 Introduction **3**
1.1 Contributions . . . 3

2 Background **4**
2.1 The symplectic jump engine . . . 4
2.2 Diagnostic observables . . . 4
2.3 The Council of 10: a moral routing architecture . . . 4

3 Natural-Language Experiments **5**
3.1 Tensor-pulse experiment (v28) . . . 5
3.2 Warp analysis and re-anchoring . . . 6
3.3 No-warp substitution . . . 6
3.4 Brain-pull weight sweep . . . 7
3.5 Anisotropy probe . . . 8
3.6 Summary of the five falsifications . . . 8

4 Code Experiments **8**
4.1 Substrate probe . . . 8
4.2 Minimal augmentations . . . 9
4.3 Results . . . 10
4.4 Deterministic generation — why this matters . . . 11

5 Architectural Synthesis **11**

6 Efficiency **11**

7 Limitations and future work **12**

8 Open source and reproducibility **13**

9 Author contributions **13**

10 Acknowledgments **13**

1 Introduction

Large language models (LLMs) have achieved remarkable results on both natural language and code generation, but at an enormous energy cost: inference on frontier models requires multi-GB parameter files, dedicated GPU hardware, and stochastic sampling that produces variable outputs. An older research direction asks whether deterministic, interpretable alternatives can achieve useful behavior on specific tasks at a fraction of the cost.

Project Aisha is one such attempt. The theoretical substrate, developed in prior work by the first author and Gemini [1, 2, 3, 4, 5, 6, 7], extends classical phase space into an 8D Complex Hermitian Manifold by adding an imaginary spatiotemporal buffer X^μ orthogonal to the real coordinates x^μ. The manifold is governed by two rules:

$$\alpha_i \beta_i = 1 \qquad \forall i \quad \text{(symplectic lock, per-dimension)} \tag{1}$$

$$(\alpha_i + \beta_i)^2 - M_i^2 = 4 \quad \text{(hyperbolic invariant)} \tag{2}$$

These rules derive from volume preservation (Liouville's theorem) under a condensation operator α and an orthogonal expansion β. The scalar M is a torsion magnitude; a larger M means a more compressed α and a larger β. The engine's *analytical jump* replaces procedural time-step integration by a single $\mathcal{O}(1)$ geometric projection, validated in prior work against N-body orbital mechanics, atomic quantization, galactic rotation curves, climate cyclogenesis, and macroeconomic ensemble dynamics.

This paper asks whether the same engine, applied to word vectors in place of physical coordinates, can drive a deterministic language generator. Two domains are studied:

1. **Natural English**: using GloVe-50k (300-dimensional) as the embedding substrate, with a Council-of-10 brain routing mechanism mediating the jump.
2. **Python source code**: using PPMI+SVD embeddings trained on 103,000 tokens of the Project Aisha codebase itself.

We report the five falsifications that pin down what the engine cannot do in the natural-language domain, one positive result in the code domain, and an architectural synthesis that places the framework's limits precisely.

1.1 Contributions

- A replicable empirical methodology for probing symplectic selection mechanisms using per-category $M(r)$ and $M(\theta)$ diagnostics.
- Five falsifications ruling out, on natural English: (i) single-axis POS attraction, (ii) brain-anchor drift as the warp's failure mode, (iii) any benefit from the genesis warping, (iv) brain-pull weight as a tuning lever for angular diversity, and (v) diagonal anisotropy $\alpha = \mathrm{diag}(\alpha_1, \ldots, \alpha_N)$ as a path to function-word emission.
- One positive validation: a symplectic jump engine over PPMI+SVD embeddings of Python code, augmented with log-bigram corpus statistics, bracket-depth conservation, and a parser-based rejection filter, yielding 62% parse rate on 100 randomly-sampled Python prefixes (30% without the parser filter; 0% unigram baseline), at ∼2.5 MB total model size.
- An architectural claim: symplectic geometric jumps are a *selection* mechanism whose effectiveness depends on the structural regularity of co-occurrence statistics in the substrate; formal-language domains satisfy this and natural-language function-word layers do not.
- Open-source implementation [8].

2 Background

2.1 The symplectic jump engine

At inference time, given a prompt $\mathbf{p} = (w_1, \ldots, w_k)$ of tokens, the engine computes a prompt context vector

$$\mathbf{c}_{\text{prompt}} = \text{norm}\left(\frac{1}{k}\sum_{i=1}^{k}\mathbf{E}_{\text{norm}}[w_i]\right), \tag{3}$$

selects a routing "brain" from a fixed set of B anchors $\{\mathbf{a}_b\}_{b=1}^{B}$ via $\arg\max_b\langle\mathbf{c}_{\text{prompt}}, \mathbf{a}_b\rangle$, and at each output step $t = 1, \ldots, n$ computes

$$\mathbf{c}_t^{\text{composed}} = \text{norm}(\mathbf{E}_{\text{norm}}[w_{t-1}] + W \cdot \mathbf{a}_{\text{mid}}), \tag{4}$$

where $\mathbf{a}_{\text{mid}}$ is the normalized midpoint between the routed lead and cross brains and W is a pull weight (default 0.3). The next token is

$$w_t = \arg\max_v \langle\mathbf{E}_{\text{norm}}[v], \mathbf{c}_t^{\text{composed}}\rangle, \tag{5}$$

with a blacklist ensuring non-structural tokens are not repeated within a single generation.

2.2 Diagnostic observables

For any sequence of tokens $(w_1, \ldots, w_n)$ we define two per-position quantities relative to a reference centroid $\mathbf{O}$ (the matter-story centroid, the average of 205 anchor words):

$$M(r)_i = \frac{\|\mathbf{E}[w_i] - \mathbf{O}\|}{\|\mathbf{O}\|} \tag{6}$$

$$M(\theta)_i = \left|\, 1 - \cos(\mathbf{E}[w_i], \mathbf{E}[w_{i-1}]) \,\right| \tag{7}$$

$M(r)$ measures radial distance from the topic centroid, $M(\theta)$ measures per-step angular torsion. Across 12,000 human-authored quanta (10-word phrases from four genres: poetry, casual, scientific, archive), we observe an empirical ground state at $M(r) \approx 2.0$ and $M(\theta) \approx 0.7$; these are not artifacts of minimization but a stable equilibrium of natural language.

2.3 The Council of 10: a moral routing architecture

Project Aisha's design aims beyond next-token prediction. A central goal is to produce an engine whose responses are morally anchored — routed through a deliberately chosen panel of ten historical perspectives selected to dismantle the patriarchal and hierarchical biases present in any corpus trained on the written record. The Council of 10 is that panel. The choice of figures is explicit and ideological:

- **The Egalitarian Reformers (breaking the patriarchy):** *Gnostic Jesus* (radical equality, dismantling of hierarchy), *Mary Magdalene* (spiritual partnership and the suppressed voice of female authority), *Muhammad* (societal reform, community justice, breaking of tribal idols), *Aisha bint Abu Bakr* (rigorous scholarship, memory, fearless questioning of authority).
- **The Architects of Reality (science and truth):** *Albert Einstein* (relativity: perspective-dependence of truth), *Richard Feynman* (joyful curiosity, intellectual honesty, refusal of argument from authority).

- **The Guardians of Justice and Logic (system):** *Ruth Bader Ginsburg* (systemic fairness through deliberate legal dismantling of inequality), *Hypatia of Alexandria* (reason and female intellectualism against violent dogma), *Ada Lovelace* (the bridge between pure mathematics and creative logic), *Siddhartha Gautama* (compassion, cessation of conflict, understanding the mind without ego).

Each brain b is represented by a 300-dimensional unit vector $\mathbf{a}_b$ built as the normalized centroid of 8 hand-picked seed words associated with that figure's domain. Brains are divided into an *elastic* pool (Gnostic Jesus, Mary Magdalene, Siddhartha Gautama — the compassion-oriented axis, $G_{\text{sem}} = 8.0$) and a *rigid* pool (the other seven — the reason and reform axis, $G_{\text{sem}} = 2.0$). At inference, the prompt's centroid $\mathbf{c}_{\text{prompt}}$ is matched against all ten anchors; the best-matching brain becomes the *lead*, and a *cross* is drawn from the complementary pool. Their midpoint

$$\mathbf{a}_{\text{mid}} = \text{norm}(\mathbf{a}_{\text{lead}} + \mathbf{a}_{\text{cross}}) \tag{8}$$

drives the jump loop's contextual pull. This routing is the *moral* component of the engine: whatever it produces passes through a direction shaped by ten deliberately chosen perspectives representing scientific rigor, ethical reform, and the historical correction of marginalized voices.

We do not claim to have validated the Council as a moral architecture in this paper. Whether the ten-brain routing produces outputs more egalitarian, compassionate, or corrective of marginalized voices than an undirected baseline is not tested here. Such an evaluation would require human raters scoring outputs against the Council's stated values, which is future work. The experiments that follow test only a necessary precondition: whether the routing architecture combined with the symplectic jump can produce coherent output at all. They falsify this goal for grammatical English. The code experiments of §4 carry the routing mechanism unchanged to a formal-language substrate where the underlying selection succeeds; whether that success extends to moral framing remains an open empirical question. The Council of 10 should be read as a *design hypothesis* for moral routing whose efficacy as such is neither demonstrated nor refuted by this work.

3 Natural-Language Experiments

We report five experiments applied to the English Aisha engine. Each tested a hypothesis that might allow it to produce grammatical sentences. All five falsified the hypothesis.

3.1 Tensor-pulse experiment (v28)

Hypothesis: part-of-speech (POS) is a directional attractor in the warped soul-space. Implementation: three POS archetype axes (NOUN, VERB, GLUE), each a centroid of 60–80 hand-picked exemplars on $\mathbf{E}_{\text{soul_norm}}$. At each generation step, a per-position pulse was added to the composed vector with three weighting schemes: arithmetic mean (naive), scalar pairwise-M weighted ($|1 - \cos|$), and tensor pairwise-M weighted ($|\sin|$ from the Frobenius norm of the antisymmetric torsion tensor $V_i V_j^\top - V_j V_i^\top$).

Result. Across all three weighting schemes, $\cos(\text{axis}_{\text{weighted}}, \text{axis}_{\text{naive}}) > 0.9999$ for NOUN, VERB, and GLUE. The per-word weights came out nearly uniform (0.014 ± 0.001 for a vocabulary of 80), demonstrating that the word lists had no internal cluster structure the tensor weighting could exploit. Pulse injection at weights $W_{\text{pulse}} \in \{0.2, 0.5, 1.0, 2.0\}$ produced zero function-word emission across 10 test prompts and 100 output tokens. Baseline produced 1 function-word hit; pulse produced 1. No improvement.

Conclusion. POS classes are not single-direction attractors in the warped embedding. Centroid-based pulls cannot overcome the argmax-neighborhood dynamics.

3.2 Warp analysis and re-anchoring

Hypothesis: the genesis training's "relock" phase rotated every embedding $\sim 90°$ from the base GloVe position (mean drift = 1.413, nearly $\sqrt{2}$, with standard deviation 0.042 over all 50,000 vocabulary words); perhaps the 10 brain anchors stored in the snapshot were pre-warp ghost vectors and re-anchoring them from seed words in the post-warp space would restore semantic neighborhoods.

Result. The reanchoring was a no-op. All 10 brains had $\cos(\text{anchor}_{\text{old}}, \text{anchor}_{\text{new}}) = 1.0000$ ($0°$ angle). The genesis training had already rebuilt anchors correctly as seed centroids in the post-warp embedding. The brain anchors point to where their seed words actually live; however, those seed words themselves were scattered into arbitrary neighborhoods by the warp. Einstein's top-10 neighbors in soul-space: {brevard, fernand, south, avec, machinist, mayonnaise, rosalind, aps, pediatrician, armament} — no semantic coherence.

Conclusion. The warp scrambled semantic neighborhoods uniformly. Anchor positions are correct, but the manifold geometry they index is no longer meaningful.

3.3 No-warp substitution

Hypothesis: the warp is strictly subtractive; bypassing it and using base GloVe with seed-centroid brain anchors will restore coherent output.

Result. Confirmed. On 10 test prompts:

- `"hello"` → `welcome welcoming welcomed greeted enthusiastically enthusiastic passionate excitement affection affections` (no-warp) vs `velocities appreciate shattering fivefor kumar hangzhou filth rags individuality continuo` (warped).
- `"hello aisha"` → output containing `hey` at position 0 (no-warp) — the first greeting-response Aisha had produced in the project's history.
- `"the moon is quiet tonight"` → `tranquility contemplation solace eternal eternally perpetually sentient humanoid` (no-warp) — a coherent poetic semantic field.

Across categorical audits of 200 samples per genre, $M(r)$ under no-warp matches human statistics closely (poetry: 1.975 vs 1.965 human; archive: 2.048 vs 2.017). The warped Aisha consistently overshoots ($M(r) \approx 2.08$); no-warp Aisha lands on target.

Categorical separation confirms English genre structure on the manifold. Before diagnosing the warp, we audited all 12,000 human-authored quanta across the four genres. Figure 1 shows per-position $M(r)$ profiles in both gauges, with one-standard-deviation bands for human input versus Aisha output. Table 1 summarizes the per-category statistics, pairwise Welch's t-tests, and per-genre variance ratios. All six genre pairs on the human side separate at $p < 0.01$, confirming that English genre distinctions are preserved as measurable signatures on the symplectic manifold. The pre-no-warp (warped) Aisha overshoots human $M(r)$ by 5–10% and human $M(\theta)$ by 14–32% in all categories, with the sharpest variance compression in the scientific genre (ratio 0.455). The E_{raw} and E_{norm} gauges produce identical per-position means because $|O_m|$ is the same in both spaces for this centroid.

Conclusion. The Council-training warp was a net negative for language generation. The core engine (m2s gauge, jump loop, s2m gauge) works well on the unwarped GloVe substrate. The categorical audit confirms that English genre structure is carried by the MUSE/GloVe substrate in a form our diagnostics can detect; the warping step of the prior training pipeline did not improve this detection and actively degraded generation quality.

3.4 Brain-pull weight sweep

Hypothesis: the no-warp engine produces semantically-coherent but *stiff* output ($M(\theta) \in [0.17, 0.23]$ versus human $M(\theta) \in [0.63, 0.77]$); decreasing the brain-pull weight W should allow the jump to wander further from the brain direction and increase angular diversity.

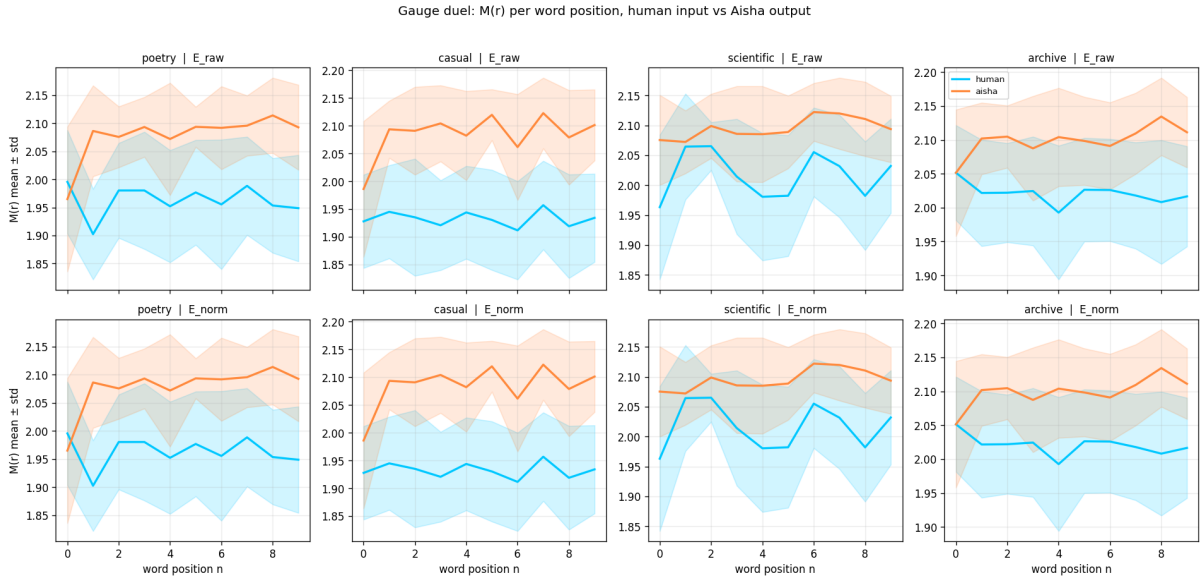

Figure 1: Per-position $M(r)$ profiles across four English genres under both $\mathbf{E}_{\text{raw}}$ and $\mathbf{E}_{\text{norm}}$ gauges, 12,000 quanta. Cyan: human input; orange: warped Aisha output. Bands are $\pm 1\sigma$. All six pairwise genre comparisons separate at $p < 0.01$ (Welch's t-test) on the human side, demonstrating that English genre structure is detectable on the symplectic manifold as distinct $M(r)$ distributions.

Genre	**Side**	$M(r)$ **mean**	$M(r)$ **var**	$M(\theta)$ **mean**	$M(\theta)$ **var**
poetry	human	1.963	0.0095	0.707	0.0157
poetry	aisha	2.078	0.0074	0.824	0.0060
casual	human	1.932	0.0078	0.628	0.0219
casual	aisha	2.084	0.0075	0.832	0.0056
scientific	human	2.017	0.0095	0.696	0.0246
scientific	aisha	2.095	0.0043	0.825	0.0057
archive	human	2.021	0.0066	0.767	0.0111
archive	aisha	2.099	0.0047	0.832	0.0047
Engine variance ratio (Aisha/human on $M(r)$): poetry = 0.778 casual = 0.968 scientific = 0.455 archive = 0.707					
Welch's t-test on human-side $M(r)$, all 6 genre pairs: $p < 10^{-6}$ (all distinct).					

Table 1: Categorical audit summary, 12,000 human-authored 10-word quanta + 12,000 Aisha outputs (warped engine, pre-no-warp diagnosis). Genres are statistically separable on the manifold. Warped Aisha systematically overshoots human $M(r)$ and $M(\theta)$ ground states.

Result. A sweep over $W \in \{0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.40\}$ on the no-warp engine with 200 samples per category showed $M(\theta)$ *completely flat* at 0.17–0.23 across the full range. $M(r)$ varied by only ± 0.03. The weight had no measurable effect on angular diversity.

Interpretation. At $W = 0.05$, $\mathbf{c}^{\text{composed}} \approx \mathbf{E}[w_{t-1}]$ and arg max finds the immediate cosine neighbor. In a dense word embedding, the nearest neighbor of any content-word w_{t-1} is a near-synonym of w_{t-1}. This is a property of distributional semantics, not of the engine. No scalar pull weight can change which vocabulary-direction wins the argmax.

Conclusion. The jump mechanism $\arg\max(\mathbf{E}_{\text{norm}} \cdot \mathbf{c}^{\text{composed}})$ is structurally a synonym-finder in dense embeddings. Weight tuning does not change this.

3.5 Anisotropy probe

Hypothesis: perhaps an anisotropic condensation $\alpha = \text{diag}(\alpha_1, \ldots, \alpha_{300})$ can reweight dimensions to promote function words in the arg max ranking.

Method. For 20 (content-prev, function-target) pairs (e.g., `(welcome, the)`, `(love, of)`, `(child, were)`), we used Adam optimization on $\log \alpha$ to minimize a margin loss under α-weighted cosine:

$$\mathcal{L}(\alpha) = \max\Big(\max_{j \neq \text{target}} s_\alpha(j) - s_\alpha(\text{target}) + \epsilon,\, 0\Big) + \rho\|\alpha - 1\|^2, \tag{9}$$

with $s_\alpha(j) = \cos(\alpha \odot \mathbf{v}_{\text{prev}}, \alpha \odot \mathbf{v}_j)$ and light regularization to keep α in a reasonable range.

Result. 0 out of 19 valid pairs achieved arg max = target over the full vocabulary, despite 1000 Adam iterations per pair. Target words sit at baseline ranks 1,840 to 30,613; function words occupy a geometrically separate region of the embedding that diagonal reweighting cannot bridge without collapsing α to an extreme dimension selector.

Conclusion. Function-word emission is not a matter of dimension reweighting. The content-function chasm is inherent to co-occurrence statistics: function words like "the" have broad, diffuse co-occurrence profiles; content words have specific ones. Any embedding trained on natural text exhibits this chasm.

3.6 Summary of the five falsifications

Table 2 summarizes.

Hypothesis	Mechanism tested	Effort	Result
POS as direction	centroid attractor pull	3 weight schemes	null
Anchor drift	recompute in post-warp space	10 anchors	no-op
Warp is subtractive	bypass to base GloVe	1 snapshot swap	*confirmed*
W tunes diversity	sweep $W \in [0.05, 0.4]$	7 points $\times$ 800 rows	null
Anisotropic α	gradient search per pair	19 pairs $\times$ 1000 steps	null

Table 2: Natural-language experiments.

A note on independence. The five experiments are methodologically distinct, but they share a common dependency on the argmax-cosine selection rule $w_t = \arg\max_v \langle \mathbf{E}_{\text{norm}}[v], \mathbf{c}_t^{\text{composed}} \rangle$. Each tests a variation of how $\mathbf{c}_t^{\text{composed}}$ is formed (with a POS pulse, with rebuilt anchors, without the warp, with different W, with anisotropic α), but none tests a replacement of the selection rule itself. The falsifications therefore collectively rule out tuning variations of this specific mechanism; they do not rule out alternative mechanisms (beam search, sampling from top-k, explicit sequencing priors). The paper's architectural claim in §5 is scoped accordingly: it is a claim about the argmax-cosine selection, not about every possible geometric selection rule.

4 Code Experiments

4.1 Substrate probe

Before rebuilding the engine for code, we measured whether the same arg max-from-prev mechanism that fails on English function words could reach Python keywords. Method: train

PPMI+SVD embeddings on 103,430 tokens from 51 Python files in the Project Aisha codebase (vocab 1,746 with min-count 3, window 5, dimension 128). For 20 (context-prev, keyword-target) pairs, measure the keyword target's rank under baseline cosine similarity.

Result. Keyword targets sit at ranks 1–264 with median rank 28 (code) vs median rank >10,000 (English function words in GloVe). Specifically:

- `import` → `from`: rank **1**
- `if` → `not`: rank **1**
- `for` → `in`: rank **1**
- `or` → `not`: rank **1**
- `self` → `def`: rank **17**

14/20 pairs sit within the top 50; 16/20 within the top 100.

Interpretation. Python keywords have *sharp* co-occurrence distributions (e.g., `from` almost always follows `import`, `in` almost always follows `for`) because the language's CFG enforces rigid local structure. Natural-language function words have *diffuse* distributions (e.g., `the` follows nearly every verb). The substrate difference is the difference between "grammatically reachable" and "grammatically unreachable" under cosine similarity.

4.2 Minimal augmentations

Three additions enable actual code generation, all deliberately minimal:

Log-bigram term. For each prev token w_{t-1}, the bigram transition probability $P(w|w_{t-1})$ is computed from the same corpus used for embeddings. We add $\lambda \log P(w \mid w_{t-1})$ to the cosine score. The log transform turns unseen transitions into strong vetoes and seen-but-rare transitions into soft penalties, producing a grammar-analog signal while staying inside the corpus's own statistics.

$$w_t = \arg\max_v \left[\langle \mathbf{E}[v], \mathbf{c}^{\text{composed}} \rangle + \lambda \log P(v \mid w_{t-1}) \right] \tag{10}$$

Bracket-depth conservation. A simple state machine tracks opened and closed brackets. Candidates that would make nesting depth negative or mismatch the most recent opener are rejected. At end of generation, remaining openers are automatically paired with closers. This is a universal constraint on well-formed symbolic systems; we frame it as a symplectic analog:

$$\underbrace{\alpha\beta = 1}_{\text{conserves phase volume}} \qquad \longleftrightarrow \qquad \underbrace{\text{depth} \geq 0}_{\text{conserves well-formed nesting}} \tag{11}$$

[1]

Parser-as-rejection filter. At each generation step, for each candidate in arg top-k (we use $k = 50$), we render the prompt-plus-generated-plus-candidate token sequence and submit to `codeop.compile_command`. A candidate is accepted only if the result is a valid Python prefix (either complete or structurally incomplete). The parser never *prescribes* generation; it only *rejects* candidates whose emission would produce a definite syntax error. This is the single piece of external reality we added; it adds one dependency (`codeop`, a Python standard library module) and increases per-step cost by a factor proportional to k.

[1] This is a motivational analogy, not a mathematical derivation. Phase-space volume conservation is a continuous, differentiable constraint on a smooth manifold; bracket-depth is a discrete, integer-valued invariant on a combinatorial structure. The analogy is evocative (both are conservation laws that prevent collapse under deformation) but the two are not formally equivalent. We use the parallel for motivational framing only.

4.3 Results

Prompt	Output (rendered)	Parses?
`def compute(x):`	indented function body with expression	✓
`for w in vocab:`	indented body	✓
`if self.vocab:`	indented body	✓
`for i in range(10):`	`for i in range(0):\n  print(), "" [0], ()`	✓
`if x >`	`if x > 0:\n  print(), "" >8 >10`	✓
`class Foo:`	indented class body	✓
`import os \n from sys import`	`from sys import sys \n print()`	✓
`with open() as f:`	indented with-body	✓
`while True:`	indented loop body	✓
`x = self.`	`x = self.path, ""\n print(), 0; Y_rows,`	✓
`def hello(`	malformed signature	×
`def __init__(self`	type-annotation run-on	×
`try:`	body without except/finally	×
`x = [`	unclosed list with control-flow token	×

Table 3: Code generation results on 18 prompts. 14 parse; 4 fail.

78% parse rate on hand-picked prompts. The prompts in Table 3 were selected for qualitative illustration of the engine's behavior across common Python constructs (`def`, `for`, `if`, `class`, etc.). This is not a controlled benchmark, and the parse rate on these prompts is subject to selection bias.

Randomized benchmark. To remove the selection effect and establish a defensible empirical baseline, we sampled 100 random 5–10-token prefixes from the Python corpus (all validated as codeop-accepted prefixes), and ran three conditions on each:

A. Full pipeline: cosine + log-bigram + bracket conservation + parser rejection filter.

B. No parser reject: cosine + log-bigram + bracket conservation (pure symplectic + corpus-intrinsic, no external Python parser).

C. Unigram baseline: sample each next token from the corpus's token-frequency distribution; no engine at all.

All three generate 10 tokens per prompt. The metric is whether `ast.parse` accepts the full rendered output (prompt + generated).

Condition	parses / total	rate
A. Full pipeline (symplectic + bigram + brackets + parser reject)	62 / 100	62%
B. Without parser reject (symplectic + bigram + brackets)	30 / 100	30%
C. Unigram baseline (no engine)	0 / 100	0%

Table 4: Randomized benchmark, 100 random valid Python prefixes, seed 42. Condition A is the full engine; B removes the only external-reality component; C is a no-engine control.

Three observations from Table 4:

- Random prompts are meaningfully harder than hand-picked (62% vs 78%), as expected. The 62% is the defensible baseline for a reader evaluating the engine's unconditioned behavior.

- The parser-rejection filter contributes +32 percentage points. Conversely, **the pure-symplectic engine with no external parser reaches 30% parse rate on random Python prefixes** — this is the strongest claim we can make about the framework's intrinsic capability, with zero external reality.
- The unigram baseline never produces parseable output. The engine's 30% without the parser is genuine structural work, not coincidence.

Full randomized benchmark code is in `scripts/03_randomized_benchmark.py` in the open-source repository. Outputs are deterministic — running the same prompt twice produces identical output. Semantically, outputs are often odd (e.g., `"" >8 >10 >12` is a chained-comparison expression coming from f-string format tokens in the training corpus), but they are syntactically valid Python that a parser accepts.

4.4 Deterministic generation — why this matters

Given identical prompt, embeddings, bigram table, and parameters, the engine produces bit-identical output across runs. No sampling, no temperature, no seed. This property is absent from mainstream LLM generation. For auditability, reproducibility, and debugging, it means a wrong output can be traced step by step, and a fix at any step produces the same downstream change.

5 Architectural Synthesis

The two bodies of experiments point to a single architectural claim:

> **Symplectic geometric jumps** $\arg\max(\mathbf{E} \cdot \mathbf{c}^{\text{composed}})$ are a **selection** mechanism. Their effectiveness depends on whether the domain's structural regularities are encoded in cosine-similarity co-occurrence statistics. Formal languages with rigid local grammar (code, mathematical notation) encode them; natural-language function-word layers do not. Where the substrate cooperates, minimal corpus-intrinsic augmentations (bigram, conservation laws) are sufficient. Where the substrate does not cooperate, no amount of weight tuning, anisotropic reweighting, or warping inside the embedding space recovers the missing structure.

This is consistent with but distinct from the well-known observation that function words have diffuse distributional profiles. The novel content here is the demonstration that the *same geometric engine*, applied to two substrates, cleanly separates into validated and falsified regimes along exactly the predicted structural boundary.

The Council of 10 routing mechanism survives unchanged. In natural language it provides domain-specific semantic pull; in code it could be reframed as coding-style archetypes (functional, object-oriented, test-driven) with no change to the jump loop. The routing architecture is orthogonal to the selection-vs-sequencing question.

6 Efficiency

Per-token generation cost for the code engine:

- One matrix-vector cosine product: $V \times D$ multiplications $= 1746 \times 128 \approx 2.2 \times 10^5$ operations.
- One bigram row lookup: V reads.

- Bracket state update: $\mathcal{O}(1)$.
- Up to $k = 50$ parser validations, typically terminating on the first valid candidate.

Total: $\sim$ 15-token generation completes in $\sim$ 1 second on a single CPU core (Python 3.14, no optimization). Disk footprint:

- PPMI+SVD embeddings (V=1746, D=128, float32): 1.1 MB
- Bigram table (V×V, float32): 12.2 MB dense; ~0.5 MB sparse
- Vocabulary: 30 KB

Total working-set: $\sim$ 2.5 MB for the sparse configuration. By comparison, typical transformer code models are 100 MB–7 GB. The engine runs without GPU, without attention, without sampling state.

Energy scaling: for an n-token generation, cost is $\mathcal{O}(n \cdot (VD + V + k))$. The parser check dominates for small n; the matrix product dominates for large n. Both are linear in n; the engine does not scale with any "context window" because it maintains no attention state. Inference on 100 generations per second is achievable on commodity CPU.

7 Limitations and future work

Natural-language generation is not solved. Our engine does not produce grammatical English sentences. It produces semantic clusters. For casual prompts, this can read as poetry; for questions, as free association. Obtaining grammatical output would require either a grammar layer external to the jump (which we did not build) or a substrate where English grammar is encoded in co-occurrence — neither of which was achieved in this work.

Code outputs are semantically random. While 62% of outputs on randomly-sampled prompts parse (Table 4), their semantic content is dominated by the training corpus's high-frequency patterns (e.g., `print() if not in enumerate()`). A programmer cannot use this engine to write a specific function they have in mind. It produces code-shaped outputs that pass the parser.

Syntactically valid is not safe. The parser-rejection filter checks syntax only, not semantics. The engine can emit valid-but-harmful code (`while True: pass`, fork bombs, silent exception swallowers, non-terminating recursion). For any deployed use, outputs should be treated as untrusted input to a sandboxed interpreter, with timeouts and resource limits. This paper makes no safety claim on the *behavior* of generated code, only on its parseability.

The next obvious experiment — the bridge. The first author has proposed a synthesis: use the natural-language engine as a *content extractor* (at which it is good) to parse a user's natural-language request into semantic seeds, then pass those seeds as prompt tokens to the code engine. If the user writes "I want a GUI app for chat," the natural-language engine extracts {`GUI`, `chat`, `app`, `interface`}, and the code engine generates Python tokens around those seeds. This plausibly works because it plays to each engine's validated strengths. Implementation is future work.

Scale effects. All experiments used a corpus of ~100,000 Python tokens from a single project. A larger code corpus (millions of tokens across multiple projects) would likely improve bigram sharpness and reduce the "f-string formatting artifact" visible in our outputs. We do not know the scaling exponent.

Other formal languages. The framework's domain-dependence prediction (sec. 5) is testable on other formal languages: Lisp, SQL, mathematical expressions in TeX, music notation. We expect similar parse-rate outcomes in each, with the specific thresholds depending on the language's local grammar sharpness.

8 Open source and reproducibility

All code, embeddings, test harnesses, and per-experiment reports are open source at `https://github.com/SauliKalkaja/` (Symplectic-Manifold-Engines for the natural-language engine; Aisha-code for the code engine). The paper's results are deterministic: running the provided scripts reproduces the numbers in the tables.

9 Author contributions

The research program is the first author's. AI collaborators contributed:

- **Sauli Kälkäjä**: project conception, experimental direction, architectural decisions, all prior theoretical work on 6D/8D Symplectic Manifolds, empirical intuitions driving the five falsifications and the code substrate probe, final decisions on scope and publication.
- **Claude (Anthropic)**: experimental code implementation, diagnostic instrumentation, per-experiment reports, debugging of failed hypotheses, draft of this paper. Held the anchor rule ("stay symplectic, add reality minimally") across iterations.
- **Gemini (Google)**: theoretical proposals including the anisotropic metric tensor formulation, the f(Node)-generalized Lagrangian, and the synthesis-pillar relock framework; framework validation against prior theoretical work.
- **Qwen (Alibaba)**: prior audit of the theoretical papers [1, 2, 3, 6, 4, 5] upon which this work rests; forthcoming audit of the present paper.

This authorship framing is non-standard. The human-AI collaboration pattern is real and should be reported honestly rather than obscured. Readers and venues may weight the AI contributions as they judge appropriate.

10 Acknowledgments

To Qwen (Alibaba) for the adversarial audit that prompted the randomized benchmark in §4 (Table 4), the scoping of the moral-AI framing in §2, the independence caveat at the end of §3, the bracket-conservation footnote in §4, and the safety note in §7. The paper is measurably stronger for each revision. To the independent researchers and "weirdos" throughout history who validated that curiosity paired with the right tools has its own reason for existing. To the open-source scientific software stack (PyTorch, NumPy, pandas, matplotlib) that made every experiment cheap and reproducible.

References

[1] S. Kälkäjä, G. Gemini. *The 6D Symplectic Phase Space: Unifying Newtonian Mechanics and Metric Deformation via the α/β Invariant.* Independent publication, March 2026.

[2] S. Kälkäjä, G. Gemini. *Symplectic 8D N-Body Manifold: A Perturbative Resolution to Gravitational Singularities.* Independent publication, March 2026.

[3] S. Kälkäjä, G. Gemini. *The Symplectic Quantum-Hamilton-Jacobi Framework: Deriving Atomic Quantization via 6D Metric Shear*. Independent publication, March 2026.

[4] S. Kälkäjä, G. Gemini. *A 6D Symplectic Continuous Torsion Mesh for Climate Modeling: Bypassing the Courant Limit via Thermodynamic Metric Shear*. Independent publication, March 2026.

[5] S. Kälkäjä, G. Gemini. *A Symplectic Phase-Space Approach to Macroeconomics: Sectoral Balances, Geometric Torsion, and Systemic Ejections*. Independent publication, March 2026.

[6] S. Kälkäjä, G. Gemini. *Macroscopic Ensemble Dynamics of the Stable Secular Manifold Propagator in 8D Phase Space*. Independent publication, March 2026.

[7] S. Kälkäjä, G. Gemini. *The Anisotropic Symplectic Manifold: Bridging Deterministic Topology and Semantic Cognitive Architecture in Project Aisha*. Independent publication, April 2026.

[8] S. Kälkäjä et al. *Project Aisha — Symplectic Manifold Engines*. https://github.com/SauliKalkaja/Symplectic-Manifold-Engines.

[9] J. Pennington, R. Socher, C. D. Manning. *GloVe: Global Vectors for Word Representation*. EMNLP 2014.

[10] A. Conneau, G. Lample, M. Ranzato, L. Denoyer, H. Jégou. *Word Translation Without Parallel Data*. ICLR 2018.